Ders 22 Genişletilmiş Notu: Vize Sorularının Çözümü, Prim ve Dijkstra Algoritmaları

Ders 22 Genişletilmiş Notu: Vize Sorularının Çözümü, Prim ve Dijkstra Algoritmaları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Vize Sorularının Çözümü: Konu Konu Dersler
- 2. Kod Sorusu: Sayfa Listesi ve Algoritmanın Akışı
- 3. Minimum Kapsayan Ağaca Dönüş: Kruskal'dan Prim'e
- 4. Prim Algoritması: Birinci Örnek
- 5. Prim'in Karmaşıklığı: Matris, Liste ve Heap
- 6. Prim Algoritması: İkinci Örnek
- 7. Başka Bir Düğümden Başlamak ve MST'nin Tekliği
- 8. Prim ve Kruskal Karşılaştırması
- 9. En Kısa Yol Problemi ve Dijkstra'nın Fikri
- 10. Dijkstra: Elle Çözüm
- 11. Dijkstra: Matrisle Çözüm ve Düğüm Seçim Sırası
- 12. Sonuçları Okumak: Köşegen Tuzağı ve Ulaşılamayan Düğümler
- 13. Araştırma Soruları

1. Vize Sorularının Çözümü: Konu Konu Dersler

Dersin ilk bölümünde hoca vize sorularını ekranda paylaştığı belge üzerinden tek tek çözdü. Amacı özellikle bazı sorularda hata yapanların doğrusunu görmesiydi. Soruların metni ve adım adım çözümleri aşağıdaki **Örnek Sorular** bölümünde soru soru verilmiştir (S1–S15). Bu bölümde, soruların arkasındaki kavramları ve hocanın her soruda üzerinde durduğu noktayı toplu olarak anlatıyoruz.

1.1 Yığın ve Kuyruk

İlk iki soru yığın (stack) ve kuyruk (queue) davranışını sınıyordu. Birinci soruda bir sayı sürekli 2'ye bölünüyor, her adımdaki kalan yığına atılıyordu. Yığın son giren ilk çıkar (LIFO) mantığıyla çalıştığı için kalanlar ters sırada çıkar ve sonuçta sayının ikilik (binary) gösterimi ekrana yazılır. Hoca bu sorunun kolay olduğunu, büyük çoğunluğun doğru cevap verdiğini söyledi.

İkinci soruda elemanlar kuyruktan alınıp yığına, sonra yığından alınıp yeniden kuyruğa aktarılıyordu. Kuyruk ilk giren ilk çıkar (FIFO), yığın LIFO olduğu için bu gidiş-dönüş, elemanların sırasını tersine çevirir.

1.2 Bağlı Liste Üzerinde İşaretçi İşlemi

Üçüncü soruda `x.next = x.next.next;` işleminin ne yaptığı soruldu. Hoca 1, 2, 3, 4 düğümlerinden oluşan bir liste çizdi ve x'i 2 numaralı düğüm olarak aldı. 2 normalde 3'ü gösteriyor.

İşlemden sonra 2, 3'ün gösterdiği düğümü yani 4'ü gösterir. 3 numaralı düğüm "bypass" edilmiş, yani listeden çıkarılmış olur. Bazı öğrenciler ayrıca `free` yapılmadığını, bu yüzden düğümün belleğinin serbest bırakılmadığını da yazmıştı. Hoca bunun da doğru bir tespit olduğunu söyledi.

1.3 Heap'ten En Büyük Elemanı Silmek

Dördüncü soru, hocanın deyişiyle en çok hata yapılan sorulardan biriydi. Max heap'ten en büyük eleman silinirken kök, dizinin son elemanıyla yer değiştirir. Sonra kökteki yeni eleman aşağı doğru heapify işlemine tabi tutulur: her seviyede iki çocuk karşılaştırılır, büyük olan yukarı çıkar ve inen eleman onun yerine geçer. İnen elemandan büyük çocuk kalmayınca işlem biter. Hocanın vurguladığı hata, heapify yolunun dışında kalan düğümleri de karşılaştırmaya katmaktı. Yalnız inen elemanın geçtiği yol üzerindeki çocuklar karşılaştırılır.

1.4 Özyinelemede Durma Koşulu

Beşinci soru, temel durumu (base condition) olmayan bir özyinelemeli (recursive) fonksiyonun ne yapacağını soruyordu. Hoca şıkları tek tek eledi. Fonksiyon teknik olarak çalışır, yani "çalışmaz" doğru değildir. Derleme hatası da vermez. "Tüm koşullar sağlanana kadar devam eder" şıkkı da yanlıştır, çünkü fonksiyonu durduracak koşul zaten temel durumdur ve o yazılmamıştır. Geriye kalan doğru cevap, fonksiyonun kendini sürekli çağırması yüzünden sonsuz çalışma durumunun oluşmasıdır. Hoca, şıklarda "runtime error" ifadesi olsaydı cevabın değişebileceğini de ekledi.

> **Ek bilgi (derste anlatılmadı):** Gerçek bir C programında her özyinelemeli çağrı çağrı yığınında (call stack) yer kaplar. Temel durum olmadığında çağrılar birikir ve yığın taşması (stack overflow) ile program bir çalışma zamanı hatasıyla sonlanır. Hocanın "runtime error şıkkı olsaydı cevap değişebilirdi" demesi bu yüzdendir.

1.5 Veri Yapısı Seçimi ve Karmaşıklık

Altıncı, yedinci, sekizinci ve dokuzuncu sorular aynı fikri farklı yapılarda sordu: hangi adresi tuttuğunuz ya da verinin durumu, karmaşıklığı belirler.

- **Tek yönlü bağlı listeyle kuyruk (S6):** Yalnız ilk eklenen elemanın (kuyruğun başının) adresi tutuluyorsa, eleman çıkarmak (dequeue) $O(1)$'dir. Ama sona eleman eklemek (enqueue) için listenin sonuna kadar gidilmesi gerekir ve bu $O(n)$'dir.
- **Tüm elemanları aynı olan dizi (S7):** Seçmeli sıralamanın (selection sort) iç döngüsünde erken çıkış yoktur, her tur sonuna kadar döner. Bu yüzden $O(n^2)$ olur. Eklemeli sıralamada (insertion sort) eleman yerinde kaldığı için kaydırma olmaz, her eleman tek karşılaştırmayla geçer ve $O(n)$ olur. Bu, sıralı dizi için de geçerlidir.
- **Dairesel bağlı listenin başına ekleme (S8):** Baş (head) yerine son düğümün (tail) adresi tutulursa, son düğümün `next`'i zaten başı gösterdiği için hem başa hem sona $O(1)$ ile erişilir. Başa ekleme en kötü durumda da $O(1)$ olur.
- **İkili arama ağacına ekleme (S9):** Elemanlar sıralı gelirse (5, 6, 7, 8, 9 gibi) ağaç tek tarafa yaslanır, yani dejenere (degenerate) olur. En kötü durumda ekleme $O(n)$'dir.

1.6 Yığınla Oluşabilecek Çıktılar

Onuncu ve on birinci sorularda `mystery` adlı bir fonksiyon 1'den 5'e kadar sayıları dolaşıyor, rastgele üretilen 0 ya da 1'e göre her sayıyı ya hemen ekrana yazıyor ya da yığına atıyordu. Sonunda yığın boşaltılıyordu. Hocanın vurgusu şuydu: yığına önce giren sonra çıkar. 2 ve 4 yığına atıldıysa önce 4, sonra 2 yazılır. Bu yüzden 1 3 5 2 4 çıktısı oluşamaz, 1 3 5 4 2 oluşabilir. Hoca iki sorunun birbirinin "ayna" hâli olarak sorulduğunu ve aslında zor olmadığını söyledi.

1.7 Sıralı Dizinin Gücü

On ikinci soruda çok hata vardı. Sırasız bir A_n ve sıralı bir B_m dizisinin ortak elemanları soruluyordu. Soruda n'nin m'den çok büyük olduğu da veriliyordu. Hoca anahtar kelimelerin "sıralı" ve "sırasız" olduğunu söyledi: sıralı dizi her zaman işimizi kolaylaştırır, çünkü onda ikili

arama (binary search) yapılabilir. Sırasız dizinin elemanları tek tek alınır ve sıralı dizide ikili aramayla aranır. Karmaşıklık $O(n \log m)$ olur.

1.8 İkili Arama Ağacında Ardıl, Yükseklik ve Doluluk

On üç ile on beşinci sorular, değerleri harflerle maskelenmiş bir ikili arama ağacında (binary search tree, BST) "X düğümünden büyük ilk değer hangi düğümdedir?" diye soruyordu. Bu, sıralamada X'ten hemen sonra gelen elemandır. Hocanın verdiği kural:

- Önce düğümün sağ çocuğuna geçilir, çünkü X'ten büyük değerler sağ taraftadır.
- Sonra sola gidilebildiği kadar gidilir. En sonda varılan düğüm, X'ten büyük ilk değerdir.
- Sağ çocuğun sol çocuğu yoksa cevap sağ çocuğun kendisidir.

On altıncı soruda ağacın yüksekliği (4), on yedincide ise yükseklik bozulmadan kaç eleman eklenebileceği soruldu. Yüksekliği 4 olan bir ikili ağaçta seviyelerde en fazla $1, 2, 4, 8, 16$ düğüm bulunabilir. Eksik yerler sayılarak cevap bulunur.

2. Kod Sorusu: Sayfa Listesi ve Algoritmanın Akışı

Vizenin son sorusu bir kod sorusuydu. Ziyaret edilen sayfalar, kod numaralarına ve bakılma sayılarına göre bir bağlı listede tutuluyordu. Soru metni Örnek Sorular bölümünde (S15) verilmiştir. Hoca burada kodu satır satır yazmadı. Öğrencilerin yaklaşık yüzde 80'inin kodunu incelediğini, asıl sorunun işlerin hangi sırayla yapıldığı yani akış olduğunu söyledi. Fonksiyonlara bölmek iyi bir şeydi, ama az fonksiyonla yazmak kritik bir sorun değildi. Belirleyici olan akıştı.

2.1 Doğru Akış

Hocanın anlattığı akış şöyledir:

1. Bir sayfaya bakıldığında önce o sayfanın listede olup olmadığına bakılır. Liste başta boştur, ama doldukça her gelen kod için kontrol gerekir. Liste, `NULL` görülmediği ve düğümün kodu aranan koda eşit olmadığı sürece gezilir.
2. Sayfa bulunduysa ikinci kontrol yapılır: sayfanın sayacı K'ye ulaştı mı? Ulaştıysa düğüm listeden çıkarılıp başa taşınır ve sayacı sıfırlanır. Ulaşmadıysa yalnız sayaç bir artırılır.
3. Listenin sonuna gelindiyse (sayfa yoksa) yeni düğüm başa eklenir. Eleman sayısını tutan sayaç bir artırılır.
4. Eleman sayısı N'yi geçtiyse sondaki sayfa silinir. Bunun için sondan bir önceki düğümün `next` alanı `NULL` yapılır. Son düğümün (tail) adresi ayrıca tutularak da yapılabilir. Bunu yapan öğrenciler de vardı.

2.2 Önceki ve Şimdiki Düğümü Birlikte Tutmak

Hocanın üzerinde en çok durduğu hata şuydu: Bazı öğrenciler listeyi tararken önceki (`prev`) ve şimdiki (`current`) düğümün adreslerini tutmamıştı. Bu yüzden sayacı K olan düğümü bulduktan sonra, onu taşımak için listede bir tur daha atmışlardı. Doğrusu, tarama sırasında `prev` ve `current`'ı birlikte tutmaktır. O zaman düğümü çıkarmak tek bir atamadır: `prev->next = current->next`. Baş adresi (`head`) zaten tutulduğu için düğüm başa kolayca eklenir. Hoca, tam puana yakın alanların genelde bunu doğru yaptığını, aynı düğümü yeniden arayanların ise puan kaybettiğini söyledi.

2.3 `while` İçinde `break` Kullanmamak

Hoca, `while` döngüsü içinde `break` kullanan öğrenciler gördüğünü, bunun şaşırtıcı olduğunu söyledi.

> **Hocanın vurgusu:** `while` döngüsünün doğasında zaten adım sayısı belli olmayan işleri yönetmek vardır. Bu yüzden `while` içinde `break` kullanmayın. Döngüden çıkma koşulunu `while`'ın koşuluna yazın. `break` genelde `for` döngüsünde kullanılır. Bir fonksiyondan `return` ile çıkmakta ise sakınca yoktur. Özyinelemeli fonksiyonlarda temel durumda `return` ile çıkılması da buna örnektir.

Yukarıdaki tarama döngüsü bunun örneğidir: “sona gelmedim ve kodu bulmadım” koşulu `while`’ın kendisine yazılır, `break` gerekmez.

2.4 Sık Yapılan Diğer Hatalar

- Bağlı liste yerine dizi kullanmak. Hoca bunun olmaması gerektiğini, bu durumda puanın önemli ölçüde kırıldığını söyledi.
- N ve K değerlerini okuyup hiç kullanmamak. Oysa bütün problem bu iki değer üzerinden dönüyordu.
- Hazır kod parçalarını akışı düşünmeden “serpiştirmek”. Hoca, fonksiyon kurgusu çok iyi olmasa da akışı doğru olanların daha yüksek puan aldığını söyledi.
- Adımların sırasını karıştırmak. Önce sayfanın var olup olmadığı kontrol edilmeliydi.

Hoca soruyu algoritmik olarak zor bulmadığını, asıl işin akışı doğru belirlemek olduğunu söyledi. Ayrıca bilgisayar başında soru çözmekle kâğıt üzerinde soru çözmenin farklı olduğunu hatırlattı.

2.5 Ara Sonrası Sorulan Soru: $n \gg m$ İfadesi

İlk bölümün sonunda bir öğrenci on ikinci soruya döndü.

Soru: Bir dizi sıralı, biri sırasızdı ve birinin uzunluğu diğerinden çok büyüktü. Bu kısmı nasıl yorumlamalıydık?

Cevap: Hoca bunun bir çeldirici olduğunu söyledi. Bir dizi sırasızsa, kısa da olsa uzun da olsa, onunla ilgili özel bir algoritma çalıştıramazsınız. Sıralı dizide ise ikili arama yapabilirsiniz. Bu yüzden sırasız dizinin n elemanı tek tek alınır ve m elemanlı sıralı dizide ikili aramayla aranır. Dizilerin uzun ya da kısa olmasının bu soru için önemi yoktu.

> **Ders sonrası not:** Hoca bu cevapta karmaşıklığı “M çarpı $\log M$” diye söyledi. Kendi ilk çözümünde doğru olarak “n çarpı $\log m$” demişti. Doğrusu $O(n \log m)$’dir: n eleman için birer ikili arama, her biri $O(\log m)$.

3. Minimum Kapsayan Ağaca Dönüş: Kruskal’dan Prim’e

Aradan sonra hoca geçen haftanın konusuna döndü. Minimum kapsayan ağaç (minimum spanning tree, MST) problemini ve kapsayan ağacın (spanning tree) ne olduğunu geçen hafta konuşmuşlardı. Çözüm için iki temel yaklaşım önerilmişti:

- **Kruskal algoritması:** Kenarlar (edge) üzerinden ilerler. Kenarlar ağırlıklarına göre sıralanır ve döngü (cycle) oluşturmadan eklenebilen kenarlar tek tek eklenir.
- **Prim algoritması:** Düğümler (vertex) üzerinden ilerler. Bu haftanın konusudur.

Prim’de bir başlangıç düğümü seçilir. Sonra o ana kadar ağaca katılmış düğümlerden çıkan olası bağlantılara bakılır ve bunların en küçüğüyle yeni bir düğüm ağaca katılır. Hoca, bu bakış açısını daha sonra en kısa yol problemi için Dijkstra algoritmasında da kullanacaklarını söyledi.

Prim’de her adımda o anki en iyi seçenek alınır. Hoca bunun açgözlü (greedy) tekniğin bir örneği olduğunu hatırlattı. Burada düğümün işaretlenmiş olması önemlidir: Düğüm düğüm ilerlendiği için, bir düğüm ağaca katıldıktan sonra onunla ilgili yeniden bir hesaplama yapmamız beklenmez.

4. Prim Algoritması: Birinci Örnek

Hoca önce beş düğümlü, yönsüz (undirected) ve ağırlıklı (weighted) bir graf çizdi:

Kenar	Ağırlık
A-B	3
A-C	6
B-C	2
B-D	1

Kenar	Ağırlık
B-E	5
C-D	4
D-E	2

> **Ders sonrası not:** Hoca grafı çizerken kenarları sayarken "D'den E'ye 2, D'den E'ye 5" dedi. Çözüm sırasında ikinci kenarın B-E olduğu anlaşılıyor ("B'den gidilebilen 5"). Bir yerde de B-E için "2" dedi. Doğrusu: D-E 2, B-E 5.

Başlangıç düğümü olarak alfabetik sıraya göre A seçildi. Adımlar:

1. **Ağaçta A var.** A'dan çıkan kenarlar A-B (3) ve A-C (6). En küçüğü A-B. B işaretlenir ve ağaca katılır.
2. **Ağaçta A, B var.** Adaylar: A-C (6), B-C (2), B-D (1), B-E (5). En küçüğü B-D (1). D daha önce listede olmadığı için eklenebilir. D işaretlenir.
3. **Ağaçta A, B, D var.** Adaylar: A-C (6), B-C (2), D-C (4), D-E (2), B-E (5). B-C ve D-E ikisi de 2 ağırlıklı ve ikisi de ziyaret edilmemiş bir düğüme gidiyor. Hoca alfabetik sırayı koruyarak B-C'yi seçti. C işaretlenir.
4. **Ağaçta A, B, C, D var.** A-C (6) ve C-D (4) kenarlarının iki ucu da artık ziyaret edilmiş olduğu için aday değildir. Geriye B-E (5) ve D-E (2) kalır. En küçüğü D-E (2). E işaretlenir.

Bütün düğümler ağaca katıldı. MST'nin kenarları A-B, B-D, B-C ve D-E'dir. Hoca toplamı adım adım biriktirdi: $3 + 2 = 5$, $5 + 1 = 6$, $6 + 2 = 8$. MST'nin toplam ağırlığı 8'dir.

5. Prim'in Karmaşıklığı: Matris, Liste ve Heap

Hoca bir soru sordu: Grafı komşuluk matrisiyle (adjacency matrix) ya da komşuluk listesiyle (adjacency list) tutmak, Prim'in karmaşıklığını değiştirir mi? İki farklı yapı bize işi daha iyi hâle getirecek başka çözümler sunabilir mi?

Kısa bir bekleyişten sonra cevabı kendisi yazdı:

- Komşuluk matrisi kullanıldığında karmaşıklık $O(V^2)$'dir.
- Komşuluk listesi kullanıldığında $O(E \log V)$'ye inilebilir. Bu daha avantajlı görünmektedir.

Ardından ikinci soruyu sordu: $O(E \log V)$'yi sağlayan yapı nedir? İpucu olarak şunu verdi: Ağaca her yeni kenar eklerken elimizdeki bilgiler arasından en küçük değeri seçiyoruz. En küçük ve en büyük elemana hızlı erişmek için daha önce gördüğümüz yapı hangisiydi? Sohbet penceresinden bir öğrenci "heap" diye yazdı. Hoca cevabı doğruladı: Komşuluk listesiyle birlikte min heap, yani ikili heap (binary heap) kullanmak bu avantajı sağlar. Her turda küçükten büyüğe dizilmiş yapıdan en küçük eleman alınır. Yerine konan eleman için yapılan heapify işlemi, karmaşıklığa log çarpanını getirir. Bu yüzden komşuluk listesi tarafında $O(E \log V)$ elde edilir.

> **Ek bilgi (derste anlatılmadı):** Matrisli gerçeklemede her adımda henüz ağaca girmemiş düğümlerin en küçük bağlantı değerleri tek tek taranır. Bu bir adımda $O(V)$ iş demektir. V adım yapıldığı için toplam $O(V^2)$ çıkar. Heap'li gerçeklemede ise her kenar en fazla bir kez heap'e girer ya da heap'teki bir değeri günceller ve her heap işlemi $O(\log V)$ sürer.

6. Prim Algoritması: İkinci Örnek

Hoca ikinci örnek için geçen hafta Kruskal'da kullandıkları dokuz düğümlü (0'dan 8'e) grafı yeniden çizdi:

Kenar	Ağırlık	Kenar	Ağırlık
0-1	4	3-4	9
0-7	8	3-5	14
1-2	8	4-5	10

Kenar	Ağırlık	Kenar	Ağırlık
1-7	11	5-6	2
2-3	7	6-7	1
2-5	4	6-8	6
2-8	2	7-8	7

6.1 Ziyaret Listesi

Hoca buradaki kilit noktanın ziyaret edilmemiş düğümleri bir listede saklamak olduğunu söyledi. 0'dan 8'e kadar düğümler için bir liste tutulur. Başlangıçta hiçbiri ziyaret edilmemiştir (hepsi 0). Bir düğüm ağaca katılınca karşısına 1 yazılır.

6.2 0 Düğümünden Başlayarak

1. **Ağaçta 0 var.** 0'dan çıkan iki kenar var: 0-1 (4) ve 0-7 (8). İkisi de ziyaret edilmemiş düğüme gidiyor. Küçük olan 0-1 seçilir. 1 işaretlenir.
2. **Ağaçta 0, 1 var.** Adaylar: 1-2 (8), 1-7 (11), 0-7 (8). İki tane 8 var. Hoca sıralamaya göre 0-7'yi seçti. 7 işaretlenir.
3. **Ağaçta 0, 1, 7 var.** 1-7 kenarının iki ucu da ziyaret edilmiş, artık bir şey ifade etmez. Adaylar: 1-2 (8), 7-8 (7), 7-6 (1). En küçüğü 7-6 (1). 6 işaretlenir.
4. **Ağaçta 0, 1, 7, 6 var.** 0'ın ziyaret edilmemiş komşusu kalmadı. Adaylar: 1-2 (8), 6-8 (6), 7-8 (7), 6-5 (2). En küçüğü 6-5 (2). 5 daha önce ziyaret edilmediği için eklenir.
5. **Ağaçta 0, 1, 5, 6, 7 var.** Adaylar: 6-8 (6), 7-8 (7), 5-2 (4), 5-4 (10), 5-3 (14), 1-2 (8). En küçüğü 5-2 (4). 2 işaretlenir.
6. **Ağaçta 0, 1, 2, 5, 6, 7 var.** 1-2 kenarı artık aday değil. Adaylar: 2-8 (2), 7-8 (7), 6-8 (6), 2-3 (7), 5-3 (14), 5-4 (10). En küçüğü 2-8 (2). 8 işaretlenir.
7. **Ağaçta 8 de var.** 7-8 ve 6-8 elenir. Adaylar: 2-3 (7), 5-3 (14), 5-4 (10). En küçüğü 2-3 (7). 3 işaretlenir.
8. **Ziyaret edilmemiş tek düğüm 4.** 4'e 3'ten (9) ya da 5'ten (10) gidilebilir. 3-4 (9) daha kısa. 4 işaretlenir.

Beşinci adımda hoca bir an düğüm yerine kenarın ağırlığını (4) işaretledi. Bir öğrenci uyardı, hoca "Doğru söylüyorsun, teşekkür ederim" diyerek 2 numaralı düğümü işaretledi. Yukarıdaki sıra düzeltilmiş hâldir.

Devre dışı kalan kenarlar 8, 11, 7, 6, 14 ve 10 ağırlıklı kenarlardır. Seçilen kenarlar ve hocanın biriktirdiği toplam:

Kenar	Ağırlık	Biriken toplam
0-1	4	4
0-7	8	12
6-7	1	13
5-6	2	15
2-8	2	17
2-5	4	21
2-3	7	28
3-4	9	37

Bütün düğümlere toplam 37 birimlik kenarla ulaşılır. Bu, geçen hafta Kruskal ile bulunan sonuçla aynıdır.

7. Başka Bir Düğümden Başlamak ve MST'nin Tekliği

Hoca şu soruyu sordu: Biz 0'ı seçerek başladık. Başka bir düğümle başlasaydık ne olurdu? Bir öğrenci sohbet penceresinden "MST değişmez" diye yazdı. Hoca bunu görmek için aynı grafı yeniden çizdi ve bu kez 8'den başladı. Bu sefer ziyaret listesini işaretlemedi, durumu hızlıca değerlendirdi.

7.1 8 Düğümünden Başlayarak

1. 8'den 7'ye (7), 2'ye (2) ve 6'ya (6) gidilebilir. Küçük olan 8-2 (2) alınır.
2. 8 ve 2'den 3'e (7), 1'e (8), 5'e (4), 6'ya (6), 7'ye (7) gidilebilir. En küçüğü 2-5 (4).
3. Adaylar arasında 5-6 (2) en küçüğüdür. 6 alınır.
4. 6'dan 7'ye 1 ile gidilebilir. Başka 1 olmadığı için uzun uzun bakmaya gerek yoktur. 6-7 (1) alınır.
5. Adaylar: 7-0 (8), 2-3 (7), 5-3 (14), 2-1 (8). 8-7 kenarı artık geçersizdir, çünkü 7 ziyaret edildi. En küçüğü 2-3 (7).
6. Adaylar: 2-1 (8), 7-0 (8), 3-4 (9), 5-4 (10). İki tane 8 var. Hoca 2-1'i seçti.
7. 1-0 (4), 7-0 (8), 3-4 (9), 5-4 (10) arasından en küçüğü 1-0 (4).
8. Son kalan 4'e en kısa 3-4 (9) ile gidilir.

Kenar	Ağırlık	Biriken toplam
0-1	4	4
1-2	8	12
2-3	7	19
2-5	4	23
2-8	2	25
3-4	9	34
5-6	2	36
6-7	1	37

Toplam yine 37 çıktı.

7.2 Ne Değişti?

Soru: Bir öncekiyle her şey aynı mı? Farklı olan bir durum var mı?

Cevap: Bir öğrenci bir bağlantının değiştiğini, ama o bağlantının önceki turda da değişebileceğini söyledi. Hoca "aynen dediğin gibi" dedi: 1-2 ve 0-7 kenarlarının ağırlıkları eşittir (8). Önceki çözümde 0-7 seçildi, bu çözümde 1-2 seçildi. Önceki turda da diğeri seçilebilirdi.

Hoca bunun sonucunu şöyle özetledi: MST algoritmasıyla gerçekten en küçük toplamı elde ediyoruz, ama ağacın kendisi değişebilir. Bunun temel sebebi bazı kenarların ağırlıklarının eşit olmasıdır. Örneğin birinci çözümde 0'dan 7'ye ağaç üzerinde doğrudan gidiliyordu. İkinci çözümde ise 0, 1, 2, 5, 6, 7 üzerinden gidiliyor. Hoca bunun bir adım sonra Dijkstra'da kullanılacak bakış açısını da biraz gösterdiğini söyledi.

Hoca, Prim'de dikkat edilmesi gereken noktayı Kruskal ile karşılaştırarak tekrarladı. Kruskal'da önemli olan döngü oluşturmamaktı. Prim'de ise bir düğümü almak için ona giden kenar o anda en küçük aday olsa bile, o düğümün daha önce ziyaret edilip edilmediğine bakılır. Düğüm ziyaret edildiyse o kenar alınmaz. Ağaç bu şekilde büyütülür.

7.3 MST Birden Fazla Olabilir mi?

Soru: Bir öğrenci sordu: O zaman burada minimum kapsayan ağaç değişebilir ama maliyet sabit kalır mı diyoruz?

Cevap: Hoca "Evet" dedi. MST birden fazla olabilir, ama olmak zorunda değildir. Sonra öğrencilere düşünmeleri için bir soru bıraktı: Bu yalnız en az iki eşit ağırlıklı kenar olduğunda mı olur? Yoksa bütün ağırlıkları birbirinden farklı bir grafta da olabilir mi? Örneğin $8+3=11$ ve $7+4=11$ gibi, farklı kenarların toplamları eşit olduğunda iki farklı kenar ikilisi birbirinin alternatifi olabilir mi? Bir öğrenci kenar sayısı aynı olursa farklılık olabileceğini düşündüğünü söyledi. Hoca bunu kendilerinin irdelemesini istedi. Cevap derste verilmedi; Örnek Sorular bölümünde (S18) verilmiştir.

8. Prim ve Kruskal Karşılaştırması

Hoca iki algoritmanın farklarını tek tek yazdı:

Prim	Kruskal
MST'ye herhangi bir düğümden başlar.	En küçük ağırlıklı kenarla başlar.
Bir düğümü birden fazla kez gezebilir (kontrol eder).	Bir düğümü bir kez gezer.
Komşuluk matrisiyle $O(V^2)$, ikili heap ve komşuluk listesiyle $O(E \log V)$.	$O(E \log V)$
Yalnız bağlı (connected) graflarda çalışır.	Böyle bir zorunluluğu yoktur.
Yoğun (dense) graflarda daha performanslıdır.	Seyrek (sparse) graflarda daha performanslıdır.
Gerçeklemesi biraz daha zordur.	Gerçeklemesi daha kolaydır.
Öncelikli kuyruk (priority queue) kullanır.	Ayrık küme (disjoint set) kullanır.

8.1 Neden Prim Yalnız Bağlı Graflarda Çalışır?

Hocanın bu satırla ilgili sorusuna bir öğrenci Prim cevabını verdi. Hoca "Neden öyle düşünüyorsun?" diye sordu.

Cevap (öğrenci): Kruskal'da kenarları sıralayıp seçiyoruz. Prim'de ise bir düğümün komşularına bakıyoruz. Bağlı olmayan bir düğümün değerine hiç bakamamış oluruz.

Hoca "aynen" diyerek doğruladı. Prim yalnız bağlı graflarda çalışır, Kruskal'da böyle bir mecburiyet yoktur.

8.2 Neden Prim Yoğun, Kruskal Seyrek Graflarda Daha İyi?

Hoca bunun basit bir mantığı olduğunu söyledi. Yoğun graflarda kenar sayısı fazladır. Kruskal kenarlar üzerinde sıralama yapar. Kenar sayısı fazla olunca bu sıralamanın yükü artar. Bu yüzden Kruskal kenar sayısı az olan seyrek graflarda daha performanslıdır.

8.3 "Bir Düğümü Birden Fazla Kez Gezer" Ne Demek?

Aradan hemen önce bir öğrenci ikinci maddeyi anlamadığını söyledi.

Soru: "It traverses one node more than one time" maddesi kafamda canlanmadı. Ne kastediliyor?

Cevap: Hoca, Prim'de her düğüm eklenirken ziyaret edilip edilmediğini gösteren bir liste tutmak gerektiğini hatırlattı. Her turda ağaçtaki düğümlerden diğer düğümlere giden bütün olası kenarlar bir liste olarak karşımıza çıkar ve bunların en küçüğü seçilir. En küçük kenar zaten ziyaret edilmiş bir düğüme gidiyorsa alınmaz. Ağaç büyüdükçe bütün bu kenarlar yeniden gözden geçirildiği için, daha önce ziyaret edilmiş bir düğüme giden başka bir küçük kenar "o düğüme tekrar gidebilir miyim?" diye baktırır. Bu karar ancak düğümün ziyaret edilip edilmediğine bakılarak verilir. Bu yüzden Prim'de bir düğüm birden fazla kez kontrol edilir.

Aradan sonra hoca son iki satırı ekledi: Prim'in gerçeklemesi biraz daha zordur, Kruskal daha kolaydır. Prim öncelikli kuyruk, Kruskal ayrık küme kullanır.

> **Ek bilgi (derste anlatılmadı):** Ayrık küme (disjoint set, union-find) yapısı, elemanları ayrık gruplarda tutar ve iki işlem sunar: bir elemanın hangi gruba ait olduğunu bulmak (find) ve iki grubu birleştirmek (union). Kruskal bir kenarı eklemeden önce iki ucunun aynı grupta olup olmadığına bakar. Aynı gruptaysa kenar döngü oluşturur ve alınmaz, değilse kenar alınır ve iki grup birleştirilir.

9. En Kısa Yol Problemi ve Dijkstra'nın Fikri

Hoca MST probleminden en kısa yol problemine geçti. Literatürde bunun için farklı algoritmalar vardır. En bilinenlerinden biri Dijkstra algoritmasıdır. Dijkstra'da bir A noktasından diğer bütün noktalara olan en kısa yolun bulunması hedeflenir. Hoca, A'dan graf içindeki herhangi bir X noktasına olan yolların yönlü kabul edildiğini söyledi. Haritalama problemlerinde bu tür en kısa yol problemlerinin çözümünde Dijkstra'dan sıkça faydalanılır.

Prim ile farkı şudur. Prim'de de seçilen bir düğümden başlıyorduk, ama o düğümden grafın diğer bütün düğümlerine ulaşmak için toplamda en küçük ağırlıklı yapıyı buluyorduk. Dijkstra'da ise seçilen düğümden geri kalan düğümlerin her birine olan en kısa yolun peşine düşeriz. Yöntemde Prim'in "esintileri" vardır ve tek bir seferde yapılan işlemlerle A'dan diğer bütün düğümlere olan en kısa yollar bulunur.

Prim'den farklı olan önemli bir nokta daha vardır: Başlangıçta kaynak dışındaki her düğüme olan uzaklık sonsuz (∞) kabul edilir. Pratikte bu, çok büyük bir sayı verilerek işaretlenir. Daha sonra bir düğüme başka bir yoldan daha küçük bir değerle gelinirse değer bu küçük değerle değiştirilir. Bu kural şöyle yazılabilir:

$$d[v] > d[u] + w(u, v) \;\Rightarrow\; d[v] = d[u] + w(u, v)$$

Burada $d[x]$, kaynaktan x'e o ana kadar bulunan en kısa uzaklık, $w(u, v)$ ise u'dan v'ye giden kenarın ağırlığıdır.

9.1 Örnek Graf

Hoca yedi düğümlü, yönlü ve ağırlıklı bir graf çizdi:

Kenar	Ağırlık
A→B	3
A→C	6
A→D	4
B→C	2
B→E	3
C→E	2
C→F	3
D→C	2
D→F	5
D→G	7
E→F	1
F→G	1

10. Dijkstra: Elle Çözüm

Hoca önce problemi "kâğıt üzerinde elle çözseydim böyle çözerdim" diyerek çözdü. Başlangıç düğümü A'dır.

1. **A'dan çıkan yollar.** A'dan B'ye 3, C'ye 6, D'ye 4 ile gidilir. Bu düğümlerin sonsuz değerleri artık bu sayılarla değişir. E, F ve G'ye A'dan doğrudan yol olmadığı için onlar sonsuz kalır.

Hoca burada bir sonraki adımda ne olacağını sordu.

Soru: Bir sonraki adımda ne olacak?

Cevap (öğrenci): Kalan düğümlere A'dan ulaşılabilecek en kısa yol yazılarak gidilecek. Hoca hangi düğümle devam edileceğini sordu. Öğrenciyle birlikte B'de karar kılındı.

2. **B işlenir.** B'den C'ye ve E'ye gidilebilir. C'ye daha önce A'dan doğrudan 6 ile gidiliyordu. B üzerinden $3 + 2 = 5$ olur. $5 < 6$ olduğu için C'nin yeni değeri 5'tir. E'ye $3 + 3 = 6$ ile gidilir. Bir öğrencinin dediği gibi "şimdilik 6, ama değişebilir". Artık C'ye A'dan doğrudan değil, A'dan B'ye, B'den C'ye gidilerek ulaşılıyor.
3. **C işlenir.** C'den E'ye $5+2 = 7$ olur. Bu 6'dan küçük olmadığı için E güncellenmez. C'den F'ye $5 + 3 = 8$ olur. Sonsuzdan küçük olduğu için F'nin değeri 8'dir.
4. **D işlenir.** D'den C'ye $4+2 = 6$ olur. Bu 5'ten küçük değil, C 5 kalır. D'den F'ye $4+5 = 9$ olur. 8'den küçük değil, F 8 kalır. D'den G'ye $4 + 7 = 11$ olur. Sonsuzdan küçük olduğu için G'nin değeri 11'dir.
5. **E işlenir.** E'nin değeri 6'dır. E'den yalnız F'ye gidilebilir: $6 + 1 = 7 < 8$. F güncellenir ve 7 olur.
6. **F işlenir.** F'den G'ye $7 + 1 = 8 < 11$. G güncellenir ve 8 olur.

Sonuç olarak A'dan en kısa uzaklıklar: B'ye 3, C'ye 5, D'ye 4, E'ye 6, F'ye 7, G'ye 8.

Hoca bu çözümde şunu gösterdiğini söyledi: Başlangıç noktasından önce en yakın komşulara gidilir. Diğer düğümlere ilerledikçe alternatif yolların daha kısa olup olmadığı kontrol edilir ve daha kısaysa değer güncellenir. Bir düğüm işlendiğinde onun nihai en küçük değeri belirlenmiş olur ve o düğümden diğerlerine devam edilir. Bu da açgözlü bir tekniktir: optimal çözümler üst üste konarak bütün düğümlere ilerlenir.

> **Ders sonrası not:** Hoca bu elle çözümde "alfabetik sırayla ilerliyorum" diyerek C'yi D'den önce işledi. Bu örnekte sonuç değişmedi (python ile kontrol edildi). Ama algoritmanın doğru sırası, hocanın matrisli çözümde vurguladığı gibi, her adımda en küçük uzaklıklı işlenmemiş düğümdür. O sırayla D (4), C'den (5) önce işlenir. Alfabetik sıra genel olarak yanlış sonuç verebilir.

10.1 Yollar MST Oluşturmaz

Hoca burada bir MST'den söz edilmediğini vurguladı. Çünkü A'dan her X düğümüne olan yolda farklı rotalar izlenir:

Düğüm	En kısa uzaklık	Yol
B	3	A→B
C	5	A→B→C
D	4	A→D
E	6	A→B→E
F	7	A→B→E→F
G	8	A→B→E→F→G

C'ye doğrudan gitmeyiz, çünkü A→C daha uzundur. Ama B'ye ve D'ye doğrudan gideriz. Yollar birbirini kesebilir ya da değişkenlik gösterebilir.

11. Dijkstra: Matrisle Çözüm ve Düğüm Seçim Sırası

Hoca, bunu analitik bir çözüme dökecek olsak ne tür bir veri yapısı kullanmamızın uygun olacağını sordu. Çok basit ve çok sık kullanılan, şaşırtıcı olmayan bir yapı olduğunu ekledi.

Soru: Dijkstra probleminde ne tür bir veri yapısı kullanılır?

Cevap (öğrenci): Düğüm sayısı kadar bir dizi açıp başlangıç düğümüne 0, diğerlerine en büyük tam sayı değerini veririz. Sonra yollara göre güncelleme yaparız.

Hoca bunun güzel bir başlangıç olduğunu söyledi. Ama A'dan bir düğüme gidip o düğümden de başka düğümlere gitme hesabı yaptığımız için, dizi yerine matris kullanmanın daha etkili olup olmayacağını sordu ve matrisle çözdü.

> **Ders sonrası not:** Öğrencinin önerdiği tek boyutlu uzaklık dizisi, Dijkstra'nın standart gerçeklemesinde kullanılan yapıdır ve en kısa uzaklıkları bulmak için yeterlidir. Hocanın matrisi, her satırda bir adımın sonunu gösterdiği için algoritmanın adımlarını izlemeyi kolaylaştırır.

11.1 Matrisin Kurulması

Matrisin sütunları hedef düğümler (B, C, D, E, F, G), satırları ise o adımda işlenen düğümdür. A'nın kendisine gidilmeyeceği için A sütunu yoktur. Başlangıçta bütün değerler sonsuzdur.

- **A satırı:** A'dan B'ye 3, sonsuzdan küçük, yazılır. C'ye 6, D'ye 4. E, F ve G'ye A'dan doğrudan bağlantı olmadığı için sonsuz kalır.
- **B satırı:** Bir öğrenci işlemi tarif etti: A'dan B'ye 3 vardı, B'den C'ye 2 eklenir, 5 olur. Kontrol C'ye olan uzaklıkla yapılır. $5 < 6$ olduğu için C sütununa 5 yazılır. B'den D'ye yol yoktur, D 4 kalır. B'den E'ye $3 + 3 = 6$, sonsuzdan küçük, yazılır. B'den F'ye ve G'ye yol yoktur.

11.2 Kritik Soru: C mi, D mi?

Bu noktada hoca kimsenin dikkatini çekmeyen önemli bir detay olduğunu söyledi.

Soru: Bir sonraki adımda D'ye mi geleceğiz, C'ye mi? Prim'de bir sonraki düğümü alırken neye göre karar veriyorduk?

Cevap: Bir öğrenci "elimizdeki düğümlere göre" dedi. Hoca bunu açtı: Elimizdeki gidilebilir düğümler arasından en kısa uzaklıkla gidilebilene göre karar veriyorduk. A'dan sonra 3, 6 ve 4 arasından en küçüğü olan B seçilmişti. Şimdi B'den sonra C'ye 5, D'ye 4, E'ye 6 ile gidilebiliyor. Yola D ile devam edilmelidir.

> **Hocanın vurgusu:** Elle çözerken arka arkaya hesap yaptığınızda bu kısım sorun gibi görünmez. Ama algoritmaya döktüğünüzde önem kazanır. Prim'de olduğu gibi, gidilebilir düğümler içinden en kısa uzaklıktaki düğümle devam etmek zorundasınız. Açgözlü teknik uyguladığımız için en küçük ulaşılabilir düğümden devam etmezsek en optimal çözümden de devam etmemiş oluruz. Bu da yanlış hesap ve yanlış sonuç olarak geri döner.

11.3 Matrisin Tamamlanması

- **D satırı:** D'den C'ye $4 + 2 = 6$. Elimizde 5 var, 5 korunur. D'den D'ye zaten bir şey yapılmaz. D'den E'ye doğrudan gidiş yoktur. D'den F'ye $4 + 5 = 9$, sonsuzdan küçük, yazılır. D'den G'ye $4 + 7 = 11$, yazılır.
- **C satırı:** Sıradaki en küçük değer C'nin 5'idir. C'den E'ye $5+2 = 7$. E 6'ydı, güncellenmez. C'den F'ye $5 + 3 = 8 < 9$, F 8 olur. C'den G'ye yol yoktur, G 11 kalır.
- **E satırı:** Bir öğrenci matrise bakarak E ile devam edileceğini söyledi. E'nin değeri 6'dır. E'den F'ye $6 + 1 = 7 < 8$, F 7 olur. E'den G'ye yol yoktur.
- **F satırı:** En küçük değer 7 olduğu için F çağrılır. F'den yalnız G'ye gidilir: $7+1 = 8 < 11$, G 8 olur.

Değerler her satırda bir alt satıra taşınarak tablo şu hâle gelir. Kalın değer, o satırın sonunda en küçük olduğu için bir sonraki adımda işlenecek düğümün nihai uzaklığıdır. "–" işareti, işlenen düğümün kendi sütununu gösterir; hoca "D'den D'ye zaten bir şey yapmayacağız" dediği gibi bu hücrelerde işlem yapılmaz:

İşlenen	B	C	D	E	F	G
A	**3**	6	4	∞	∞	∞
B	–	5	**4**	6	∞	∞
D	3	**5**	–	6	9	11
C	3	–	4	**6**	8	11
E	3	5	4	–	**7**	11
F	3	5	4	6	–	**8**

İşlenme sırası A, B, D, C, E, F'dir. Hoca bunu şöyle özetledi: A'dan sonra B gelir. B'den sonra C değil D gelir. Ondan sonra C, sonra E, sonra F gelir. Bu sıra tamamen düğümlerin uzaklıklarına göre değişir.

12. Sonuçları Okumak: Köşegen Tuzağı ve Ulaşılamayan Düğümler

Hoca, A'dan diğer düğümlere olan en kısa yol uzunluklarını ekrana yazdırmak istesek matristen neleri yazdıracağımızı sordu. Öğrencilerden "son eleman", "sonsuzdan farklı olan", "sonsuz olmadan bir önceki eleman" gibi cevaplar geldi. Hoca matrise sağdan, soldan, çaprazdan bakmalarını söyledi ve sonunda kendisi "köşegeni yazdırırım" dedi. Köşegen yazdırılırsa A'dan diğer bütün düğümlere olan en kısa yol uzunlukları yazdırılmış olur gibi görünür.

Ardından burada "ufak bir kaçak" olduğunu, onu konuşmadıklarını söyledi.

Soru (öğrenci): Köşegen her zaman doğru oluyor mu?

Cevap: Hoca tam da bunu tartışmaya açmak istediğini söyledi. Bir öğrenci "böyle olunca D'ye ulaşılamıyor" dedi, hoca doğruladı. Matriste D satırında D'den D'ye bir işlem yapılmadığı için

köşegendeki D hücresi boş kalır ve sanki A'dan D'ye yol yokmuş gibi bir izlenim oluşur. Doğru değerler 3, 5, 4, 6, 7 ve 8'dir, ama bunlar her zaman köşegene denk gelmeyebilir. Değerlerin aşağıya doğru taşınmasının en büyük sebebi budur: Her sütunda sonsuzdan önce görülen son değer, yani o düğüm işlenirken sahip olduğu değer, en küçük değerdir ve okunması gereken odur.

12.1 Başka Bir Kaynaktan Başlamak

Hoca aynı işlemin A yerine örneğin E'den de başlatılabileceğini söyledi. O zaman bazı düğümlere hiç ulaşılamayabilir. E'den A'ya gidiş yoktur. E'den C'ye gidiş de yoktur. Kenarlar yönlü olduğu için bu durumda o düğümlerin değeri sonsuz kalır.

> **Ders sonrası not:** Altyazıda "E'den B'ye gidebilirsiniz" geçiyor. Bu graf yönlü olduğu için E'den B'ye yol yoktur. E'den yalnız F'ye (1) ve F üzerinden G'ye (2) ulaşılır; A, B, C ve D'ye ulaşılamaz (python ile kontrol edildi).

13. Araştırma Soruları

Hoca dersi, cevabı verilmeyen ve öğrencilerin araştırmasını istediği sorularla bitirdi. Bunlar Örnek Sorular bölümünde (S22–S24) çözümleriyle verilmiştir.

- Şu ana kadar yalnız en kısa yolların uzunluklarını bulduk. En kısa yolların hangi düğümler üzerinden oluştuğunu nasıl buluruz?
- İki düğüm arasındaki mesafe negatif verilebilseydi Dijkstra çözüm üretebilir miydi? Yollarda negatif mesafe olmaz, ama yönlü ve ağırlıklı bir grafta negatif ağırlık verebileceğimiz problemler var mıdır? Dijkstra bunu çözer mi, çözemezse hangi algoritma çözer?
- En kısa yol için alternatif algoritmalar nelerdir?
- En kısa yol yerine en büyük ağırlıklı yolu bulmayı gerektiren problemler var mıdır? Hoca burada "maximum flow" ifadesini kullandı.

Hoca, en kısa yol probleminin aslında dersin doğrudan konusu olmadığını, Dijkstra'yı graf algoritmalarına bir örnek olması için anlattığını söyledi. Bu soruların bazılarının ders kapsamında görüleceğini de ekledi.

Örnek Sorular

Bu bölümde dersin ilk yarısında çözülen vize soruları (S1–S15) ve derste sözlü sorulan sorular (S16–S24) yer alır. Vize soruları hocanın ekranda paylaştığı belgedeydi. Bu ders için hazır kare olmadığından soru metinleri hocanın sesli okumasından kurulmuştur. Bu yüzden hepsi "derste sözlü" olarak işaretlenmiştir.

S1. Yığınla Sayı Dönüştürme (derste sözlü, 00:16:14)

Aşağıdaki yarı kod parçası $n = 20$ için ekrana ne yazar? Program hangi işi gerçekleştirir?

```
while n > 0:
    push(n mod 2)
    n = n / 2
while yığın boş değil:
    yazdır(pop())
```

Çözüm (derste verildi): $n = 20 > 0$, $20 \bmod 2 = 0$ yığına atılır, $n = 10$. $10 \bmod 2 = 0$, $n = 5$. $5 \bmod 2 = 1$, $n = 2$. $2 \bmod 2 = 0$, $n = 1$. $1 \bmod 2 = 1$, $n = 0$ ve döngü biter. Yığının dibinden tepesine 0, 0, 1, 0, 1 vardır. Boşaltırken önce tepedeki çıkar: ekrana `10100` yazılır. Program bir sayıyı ikilik düzene dönüştürür ($20_{10} = 10100_2$).

Aynı mantığın C karşılığı (`gcc -Wall` ile derlenip çalıştırıldı, çıktı `10100`):

```c
#include <stdio.h>

#define MAX 64

int stack[MAX];
```

```c
int top = -1;

void push(int x) { stack[++top] = x; }
int pop(void) { return stack[top--]; }
int isEmpty(void) { return top == -1; }

int main(void) {
    int n = 20;
    while (n > 0) {
        push(n % 2);
        n = n / 2;
    }
    while (!isEmpty()) {
        printf("%d", pop());
    }
    printf("\n");
    return 0;
}
```

S2. Kuyruk ve Yığın Arasında Aktarım (derste sözlü, 00:18:44)

Bir kuyrukta baştan sona 5, 2, 1, 4 bulunmaktadır. Kuyruk boşalana kadar `dequeue` ile alınan eleman yığına `push` edilir. Sonra yığın boşalana kadar `pop` edilen eleman kuyruğa `enqueue` edilir. Son olarak kuyruk boşalana kadar `dequeue` edilen eleman ekrana yazılır. Çıktı nedir?

Çözüm (derste verildi): Yığına sırayla 5, 2, 1, 4 girer; dipte 5, tepede 4 vardır. `pop` sırası 4, 1, 2, 5'tir ve kuyruğa bu sırayla girerler. Kuyruğun başında 4 olduğu için çıktı **4, 1, 2, 5** olur.

S3. `x.next = x.next.next` (derste sözlü, 00:20:16)

Son düğüm olmadığı bilinen bir x düğümü için `x.next = x.next.next;` işlemi yapıldığında ne olur?

Çözüm (derste verildi): 1→2→3→4 listesinde x 2 olsun. 2'nin `next`'ine, 3'ün `next`'i olan 4'ün adresi atanır. 2 artık 4'ü gösterir, 3 atlanır. Yani x'ten sonraki düğüm listeden silinir. `free` yapılmadığı için bu düğümün belleği serbest bırakılmaz.

S4. Max Heap'ten Silme (derste sözlü, 00:21:47)

Verilen max heap ağacında en büyük elemanın silinip ağacın heapify edilmesi işleminde hangi sayılar karşılaştırma işlemlerine girer?

Çözüm (derste verildi):

1. Kökteki 38 silinecektir. Sondaki eleman 19 köke alınır.
2. 19, çocukları 37 ve 35 ile karşılaştırılır. Max heap olduğu için büyük olan 37 yukarı çıkar, 19 onun yerine iner.
3. 19, yeni çocukları 34 ve 36 ile karşılaştırılır. 36 yukarı çıkar, 19 onun yerine iner.
4. 19, çocukları 14 ve 17 ile karşılaştırılır. İkisi de 19'dan küçüktür, işlem biter.

Karşılaştırmaya giren sayılar: **19, 37, 35, 34, 36, 14, 17**. Heapify yolu dışında kalan 16 ve 15'i karşılaştırmaya katmak, hocanın belirttiği sık hatadır.

S5. Temel Durumu Olmayan Özyineleme (derste sözlü, 00:23:29)

Özyinelemeli bir fonksiyonda temel durum (base condition) bulunmuyorsa aşağıdakilerden hangisi gerçekleşir? Şıklar: fonksiyon çalışmaz; tüm koşullar sağlanana kadar devam eder; derleme hatası verir; sonsuz çalışma durumu oluşur.

Çözüm (derste verildi): Fonksiyon teknik olarak çalışır ve derleme hatası vermez. Durma koşulunu temel durum oluşturur; o yazılmadığı için "koşullar sağlanana kadar" durması da mümkün değildir. Fonksiyon kendini sürekli çağırır: **sonsuz çalışma durumu oluşur (D)**.

S6. Tek Yönlü Bağlı Listeyle Kuyruk (derste sözlü, 00:24:41)

Tek yönlü bağlı liste ile oluşturulan bir kuyrukta ilk eklenen eleman için adres bilgisi saklanırken, en son eklenen eleman için ayrıca adres saklanmamaktadır. `enqueue` ve `dequeue` işlemlerinin karmaşıklığı nedir?

Çözüm (derste verildi): 1, 2, 3, 4 sırayla eklensin. 1 kuyruğun başı, 4 sonudur. Yeni eleman (5) sona ekleneceği için baştan sona kadar gidilmelidir: `enqueue` $O(n)$. Baştaki elemanın adresi tutulduğu için 1'i çıkarmak doğrudandır: `dequeue` $O(1)$.

S7. Aynı Elemanlı Dizide Sıralama (derste sözlü, 00:26:08)

Tamamı aynı sayıdan oluşan n elemanlı bir dizi selection sort ve insertion sort ile sıralansaydı karmaşıklık ne olurdu?

Çözüm (derste verildi): Selection sort her turda en küçük elemanı bulmak için iç döngüyü sonuna kadar döner. O turda değişiklik olup olmadığına bakılmaz: $O(n^2)$. Insertion sort'ta yer değiştirme gerekmeyince kaydırma da olmaz, her eleman bir karşılaştırmayla yerinde kalır: $O(n)$. Aynı durum sıralı dizi için de geçerlidir.

S8. Dairesel Bağlı Listenin Başına Ekleme (derste sözlü, 00:27:35)

Bir dairesel bağlı listenin önüne yeni bir eleman eklemenin maliyeti en kötü durumda büyük O gösteriminde nedir?

Çözüm (derste verildi): 1→2→3→4→5→1 listesinde baş yerine son düğümün (5) adresi saklanırsa, 5'in `next`'i zaten başı gösterdiği için hem başa hem sona erişilir. Yeni eleman 6 gelince 5, 6'yı; 6 da 1'i gösterir. En kötü durumda maliyet $O(1)$ olur.

S9. İkili Arama Ağacına Ekleme (derste sözlü, 00:29:14)

Bir ikili arama ağacına eleman eklemek en kötü durumda hangi karmaşıklıktadır?

Çözüm (derste verildi): 5, 6, 7, 8, 9 sırayla gelirse her eleman bir öncekinden büyük olduğu için ağaç hep sağa dallanır. Dengesiz, dejenere bir ağaç oluşur. En kötü durumda ekleme $O(n)$ olur.

S10. `mystery` Fonksiyonunun Çıktıları (derste sözlü, 00:30:06)

`mystery` fonksiyonunda $i = 1, \ldots, 5$ için rastgele 0 ya da 1 üretilir. 0 gelirse i hemen ekrana yazılır, 1 gelirse i yığına atılır. Döngü bitince yığın boşaltılarak elemanlar yazılır. (a) 1 3 5 2 4 çıktısı oluşabilir mi? (b) 1 3 5 4 2 çıktısı oluşabilir mi? Sebebini açıklayınız.

Çözüm (derste verildi): 1, 3 ve 5 hemen yazılmış, 2 ve 4 yığına atılmış olmalıdır. Yığına önce 2, sonra 4 girer; 2 altta, 4 üstte kalır. Boşaltılırken önce 4, sonra 2 yazılır. (a) **Oluşamaz.** (b) **Oluşabilir.** Beş sayının 32 olası 0/1 dizisi python ile tek tek denendi: 1 3 5 2 4 hiçbirinde çıkmıyor, 1 3 5 4 2 çıkıyor.

S11. Sırasız ve Sıralı Dizinin Ortak Elemanları (derste sözlü, 00:32:21)

Sırasız bir A_n ve sıralı bir B_m dizisi veriliyor; n'nin m'den çok büyük olduğu biliniyor. İki dizinin ortak elemanlarını bulmak için nasıl bir yol izlersiniz? Karmaşıklık nedir?

Çözüm (derste verildi): Belirleyici olan dizilerin sıralı olup olmamasıdır; $n \gg m$ ifadesi çeldiricidir. A'nın her elemanı alınır ve sıralı B dizisinde ikili aramayla aranır. Karmaşıklık $O(n \log m)$ olur. (Hocanın aradan sonraki "$M \log M$" ifadesi için 2.5'teki ders sonrası nota bakın.)

S12. BST'de "X'ten Büyük İlk Değer" (derste sözlü, 00:33:55)

Değerleri harflerle maskelenmiş bir ikili arama ağacı veriliyor. Sıralamada (a) A düğümünden, (b) B düğümünden, (c) C düğümünden büyük ilk değer hangi düğümde yer alır?

Çözüm (derste verildi): Kural: önce sağ çocuğa geçilir, sonra sola gidilebildiği kadar gidilir.

- (a) A'nın sağ alt ağacında sola inilen en uçtaki düğüm **J**'dir.
- (b) B'nin sağ çocuğu E'dir. E'nin sol çocuğu olmadığı için cevap **E**'dir.
- (c) C'den sonra G'ye geçilir. G'nin sol çocuğu K'dır ve K'nın başka çocuğu yoktur. Cevap **K**'dır.

S13. Ağacın Yüksekliği (derste sözlü, 00:36:40)

Aynı ağacın yüksekliği nedir?

Çözüm (derste verildi): Kökten en derin yaprağa seviyeler sayılınca yükseklik **4** bulunur.

S14. Yüksekliği Bozmadan Eklenebilecek Eleman Sayısı (derste sözlü, 00:36:50)

Ağacın mevcut yüksekliğini bozmadan kaç adet eleman eklenebilir?

Çözüm (derste verildi): Yüksekliği 4 olan bir ikili ağaçta 0., 1., 2., 3. ve 4. seviyelerde en fazla 1, 2, 4, 8 ve 16 düğüm olabilir. Dördüncü seviyede şu an 1 eleman var, oraya 15 eleman daha eklenebilir. Bir önceki seviyede çocuğu eksik 3 yer var (hoca bunları tek tek saydı), oraya 3 eleman eklenebilir. Cevap $15 + 3 = 18$.

Sağlama (ara adım eklendi): Dolu bir ağaçta $1 + 2 + 4 + 8 + 16 = 31$ düğüm olur. 18 eleman eklenebiliyorsa ağaçta $31 - 18 = 13$ düğüm vardır. Bu, 0–2. seviyelerin dolu (7 düğüm), 3. seviyede 5 ve 4. seviyede 1 düğüm olmasıyla tutarlıdır.

S15. Kod Sorusu: Sayfa Listesi (derste sözlü, 00:38:27)

Bir arama motorunda ziyaret edilen sayfalar, kod numaralarına ve sayfaya bakılma sayısına göre aşağıdaki kurallara uygun şekilde bir bağlı listeye yerleştirilmektedir:

- Bir sayfaya ilk defa bakıldıysa sayfanın kod numarası bağlı listenin başına yerleştirilir.
- Bu yeni eleman eklenince bağlı listedeki eleman sayısı N'yi geçtiyse en sondaki sayfaya ait kod numarası listeden silinir.
- Daha önce bakılmış bir sayfaya bakıldığında o sayfanın sayacı bir artırılır.
- Sayaç K olduysa o sayfanın kod numarası listenin başına getirilir ve sayacı sıfırlanır.

Bağlı listenin başlangıçta boş olduğu varsayılır. N ve K değerleri dışarıdan okunur.

Çözüm (derste verildi): Hoca kodu yazmadı, akışı anlattı (bölüm 2). Önce sayfanın listede olup olmadığı `prev` ve `current` tutularak aranır. Bulunduysa sayaca göre ya yalnız sayaç artırılır ya da düğüm `prev->next = current->next` ile çıkarılıp başa taşınır. Bulunmadıysa yeni düğüm başa eklenir, eleman sayısı N'yi geçerse sondan bir önceki düğümün `next`'i `NULL` yapılır.

> **Çözüm kodu (derste verilmedi):** Aşağıdaki kod hocanın anlattığı akışa göre yazıldı. Sayaç önce artırılır, sonra K ile karşılaştırılır (soru metnindeki sıra); ilk ziyarette sayaç 1 alındı. `while` içinde `break` kullanılmadı. `gcc -Wall -Wextra` ile uyarısız derlendi; `N = 3`, `K = 3` ve 10 20 30 10 10 40 20 50 ziyaret dizisiyle çalıştırıldı. Son liste `50(1) 20(1) 40(1)` oldu: 10 üçüncü ziyaretinde başa taşındı, 40 gelince listenin sonundaki 20 silindi.

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct node {
    int kod;
    int sayac;
    struct node *next;
} NODE;

/* Bir sayfa ziyaretini isler. head: listenin basi, adet: eleman sayisi. */
NODE *ziyaret(NODE *head, int kod, int N, int K, int *adet) {
    NODE *prev = NULL;
    NODE *current = head;
```

```c
    /* 1. Sayfa listede var mi? prev ve current birlikte tutulur. */
    while (current != NULL && current->kod != kod) {
        prev = current;
        current = current->next;
    }

    if (current != NULL) {
        /* 2. Sayfa bulundu: sayac artirilir. */
        current->sayac++;
        if (current->sayac == K) {
            current->sayac = 0;
            if (prev != NULL) {
                /* Dugum bulundugu yerden cikarilir, basa eklenir. */
                prev->next = current->next;
                current->next = head;
                head = current;
            }
        }
    } else {
        /* 3. Sayfa yok: yeni dugum basa eklenir. */
        NODE *yeni = (NODE *)malloc(sizeof(NODE));
        yeni->kod = kod;
        yeni->sayac = 1;
        yeni->next = head;
        head = yeni;
        (*adet)++;
        if (*adet > N) {
            /* Sondan bir onceki dugum bulunur, son dugum silinir. */
            NODE *p = head;
            while (p->next->next != NULL) {
                p = p->next;
            }
            free(p->next);
            p->next = NULL;
            (*adet)--;
        }
    }
    return head;
}

void yazdir(NODE *head) {
    while (head != NULL) {
        printf("%d(%d) ", head->kod, head->sayac);
        head = head->next;
    }
    printf("\n");
}

int main(void) {
    int N, K, kod, adet = 0;
    NODE *head = NULL;
    if (scanf("%d %d", &N, &K) != 2) return 1;
    while (scanf("%d", &kod) == 1 && kod != -1) {
        head = ziyaret(head, kod, N, K, &adet);
        yazdir(head);
    }
    while (head != NULL) {
        NODE *t = head;
        head = head->next;
```

```
        free(t);
    }
    return 0;
}
```

S16. Prim'de Matris mi, Liste mi? (derste sözlü, 01:13:52)

Prim algoritmasında komşuluk matrisi ya da komşuluk listesi kullanmak karmaşıklığı değiştirir mi? $O(E \log V)$ karmaşıklığını hangi yapı sağlar?

Çözüm (derste verildi): Komşuluk matrisiyle $O(V^2)$, komşuluk listesiyle $O(E \log V)$. Bunu sağlayan yapı, her turda en küçük kenara hızlı erişimi veren min heap (ikili heap) yapısıdır. En küçük eleman alındıktan sonra yapılan heapify işlemi $\log V$ çarpanını getirir.

S17. Prim'e Başka Bir Düğümden Başlamak (derste sözlü, 01:25:38)

Dokuz düğümlü örnekte Prim'e 0 yerine başka bir düğümden başlasaydık ne olurdu? MST değişir mi?

Çözüm (derste verildi): 8'den başlanınca toplam yine 37 çıktı. Yalnız bir kenar değişti: 0-7 yerine 1-2 seçildi. İkisinin ağırlığı da 8 olduğu için hangisi seçilirse seçilsin toplam aynıdır. Ağacın kendisi değişebilir, maliyet değişmez. Python ile grafın bütün kapsayan ağaçları sayıldı: en küçük toplam 37'dir ve bu toplamı veren tam 2 farklı ağaç vardır (biri 0-7'yi, diğeri 1-2'yi içerir).

S18. Bütün Ağırlıklar Farklıysa MST Tek midir? (derste sözlü, 01:33:27)

En az iki eşit kenar olunca birden fazla MST olabileceğini gördük. Bütün kenar ağırlıkları birbirinden farklı bir grafta da birden fazla MST olabilir mi? Örneğin $8+3=11$ ve $7+4=11$ gibi eşit toplamlar iki alternatif MST oluşturabilir mi?

> **Çözüm (derste verilmedi):** Hayır. Bütün kenar ağırlıkları farklıysa MST tektir. Kanıt: İki farklı MST T_1 ve T_2 olsun. İkisinden yalnız birinde bulunan kenarlar arasında en hafifi e olsun, $e \in T_1$ diyelim. T_2'ye e eklenince bir döngü oluşur. Bu döngüde T_1'de olmayan bir f kenarı vardır ve f de iki ağaçtan yalnız birinde bulunduğu için $w(f) > w(e)$'dir (ağırlıklar farklı ve e en hafifi). $T_2 + e - f$ yine bir kapsayan ağaçtır ve T_2'den daha hafiftir. Bu, T_2'nin minimum olmasıyla çelişir. Toplam eşitlikleri bu kanıtı etkilemez, çünkü karşılaştırma tek tek kenarlar üzerinden yapılır. Python ile 6 düğümlü, ağırlıkları farklı rastgele graflarda bütün kapsayan ağaçlar sayıldı; hiçbirinde birden fazla MST çıkmadı.

S19. Dijkstra'da Sıradaki Düğüm: C mi, D mi? (derste sözlü, 02:13:19)

Matrisli çözümde A ve B işlendikten sonra C'ye 5, D'ye 4, E'ye 6 ile gidilebiliyor. Bir sonraki adımda hangi düğümle devam edilmelidir? Neye göre karar verilir?

Çözüm (derste verildi): Prim'de olduğu gibi, henüz işlenmemiş düğümler arasından en küçük uzaklığa sahip olanla devam edilir. Cevap **D**'dir (4). Sonraki sıra C, E, F'dir. En küçük düğümle devam edilmezse açgözlü yöntem optimal çözümü kaçırabilir.

S20. Dijkstra İçin Veri Yapısı (derste sözlü, 02:07:44)

Dijkstra'yı analitik bir çözüme dökecek olsak ne tür bir veri yapısı kullanmak uygun olur?

Çözüm (derste verildi): Bir öğrenci düğüm sayısı kadar bir dizi önerdi: başlangıca 0, diğerlerine en büyük tam sayı verilir ve yollara göre güncellenir. Hoca bunu güzel bir başlangıç saydı ve adımları izlemek için satırları işlenen düğümler, sütunları hedef düğümler olan bir matris kullandı (bölüm 11). Matrisin son hâli bölüm 11.3'teki tablodur.

S21. Sonuçlar Köşegenden Okunabilir mi? (derste sözlü, 02:19:12)

A'dan diğer düğümlere olan en kısa yol uzunluklarını ekrana yazdırmak istesek matristen neyi yazdırırız? Köşegen her zaman doğru sonucu verir mi?

Çözüm (derste verildi): Köşegen her zaman doğru sonucu vermez. Bu örnekte işlenme sırası A, B, D, C, E, F olduğu için D satırının D sütunu boş kalır ve sanki A'dan D'ye yol yokmuş gibi görünür. Doğru okuma, her sütunda sonsuzdan önce görülen son değeri, yani düğüm işlenirken sahip olduğu değeri almaktır: B 3, C 5, D 4, E 6, F 7, G 8.

S22. En Kısa Yolun Düğümleri Nasıl Bulunur? (derste sözlü, 02:25:10)

Şu ana kadar yalnız en kısa yolların uzunluklarını bulduk. En kısa yolların hangi düğümler üzerinden oluştuğunu nasıl buluruz?

> **Çözüm (derste verilmedi):** Her güncellemede ($d[v] > d[u] + w(u, v)$ olduğunda) uzaklıkla birlikte v'nin önceki düğümü de bir dizide saklanır: onceki$[v] = u$. Algoritma bitince yol, hedef düğümden başlayıp önceki düğümler izlenerek kaynağa kadar geriye doğru okunur. Dersteki örnek için python ile bulunan önceki düğümler: B←A, C←B, D←A, E←B, F←E, G←F. Örneğin G için G←F←E←B←A, yani yol A→B→E→F→G'dir.

S23. Negatif Ağırlıklar ve Diğer En Kısa Yol Algoritmaları (derste sözlü, 02:25:41)

İki düğüm arasındaki mesafe negatif verilebilse Dijkstra çözüm üretebilir miydi? Yönlü ve ağırlıklı bir grafta negatif ağırlık verilebilecek problemler var mıdır? Dijkstra bunu çözemezse hangi algoritma çözer? Alternatif en kısa yol algoritmaları nelerdir?

> **Çözüm (derste verilmedi):** Dijkstra, işlediği bir düğümün değerinin artık hiç küçülmeyeceğini varsayar. Negatif ağırlıklı bir kenar, işlenmiş bir düğüme sonradan daha kısa bir yol açabileceği için bu varsayım bozulur ve Dijkstra yanlış sonuç verebilir. Kenar ağırlığı mesafe değil de maliyet ya da kazanç gibi bir büyüklükse negatif ağırlıklar ortaya çıkabilir. Negatif ağırlıklı graflar için Bellman-Ford algoritması kullanılır; bu algoritma negatif döngüleri de tespit eder. Bütün düğüm çiftleri arasındaki en kısa yollar için Floyd-Warshall algoritması kullanılabilir.

S24. En Büyük Ağırlıklı Yol ve Maksimum Akış (derste sözlü, 02:26:30)

En kısa yol yerine en uzun yolu ya da en büyük ağırlıklı yolu bulmayı gerektiren problemler var mıdır? (Hoca burada "maximum flow" ifadesini kullandı.)

> **Çözüm (derste verilmedi):** Bunlar iki ayrı problemdir. Maksimum akış (maximum flow) problemi en uzun yolu aramaz; kapasiteli bir ağda kaynaktan hedefe aynı anda geçirilebilecek en büyük akış miktarını bulur. Ford-Fulkerson yöntemi bunun için kullanılır. En uzun yol (longest path) problemi ise ayrı bir problemdir. Yönlü döngüsüz graflarda (DAG) verimli çözülür; proje planlamasındaki kritik yol bunun bir örneğidir. Genel graflarda en uzun basit yolu bulmak için bilinen verimli (polinom zamanlı) bir algoritma yoktur.

Dersin Toparlaması

Dersin ilk yarısında vize soruları üzerinden yığın, kuyruk, bağlı liste, heap, özyineleme, sıralama ve ikili arama ağacı konuları tekrar edildi. Ortak ders şuydu: Karmaşıklığı veri yapısının kendisi kadar hangi adresi tuttuğunuz ve verinin durumu (sıralı mı, aynı mı) belirler. Kod sorusunda ise asıl önemli olan akıştı: önce sayfanın var olup olmadığına bakmak, `prev` ve `current`'ı birlikte tutarak listeyi bir kez gezmek ve `while` içinde `break` kullanmamak.

İkinci yarıda MST problemine Prim algoritmasıyla dönüldü. Prim, herhangi bir düğümden başlar ve her adımda ağaçtan dışarı çıkan en küçük kenarla ziyaret edilmemiş bir düğümü ağaca katar. Komşuluk matrisiyle $O(V^2)$, komşuluk listesi ve min heap ile $O(E \log V)$ çalışır. Eşit ağırlıklı kenarlar varsa farklı MST'ler çıkabilir, ama toplam ağırlık değişmez. Prim yoğun, Kruskal seyrek graflarda daha iyidir. Son olarak Prim'in bakış açısı en kısa yol problemine

taşındı: Dijkstra, kaynaktan başlayıp uzaklıkları sonsuzdan güncelleyerek ve her adımda en küçük uzaklıklı işlenmemiş düğümle devam ederek kaynaktan bütün düğümlere en kısa yolları bulur. Hoca, sonraki haftalarda metin arama (string search) ve sıralama algoritmalarının konuşulacağını söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.